

Actividad 1. Minería de datos

Nombre: Juan Sebastian Manrique Moreno

Código: 20202107016

Correo: jsmanriquem@udistrital.edu.co

Importación de librerías y del archivo .csv

Se importan las librerías `pandas` y `numpy` con sus alias "pd" y "np", respectivamente.

```
In [ ]: import pandas as pd
```

Ejercicio 1.

Para este ejercicio, se importarán un conjunto de datos con el listado de colegios públicos de Bogotá del año 2016.

Se guarda el DataFrame "LISTADO_COLEGIOS_BOGOTA.csv" en su respectivo objeto, llamado `df`.

```
In [ ]: df = pd.read_csv("./LISTADO_COLEGIOS_BOGOTA.csv")
```

1. ¿Cuáles son las columnas y cuántas filas tiene?

Para conocer cuáles son las columnas se puede usar el método `.columns()` de pandas, sin embargo, la forma en la que expresa cuáles son las columnas no es muy atractivo. Para cambiar la forma en la cual se muestran las columnas se usa el método `.tolist()` el cual pertenece a pandas y el método `.join()` el cual es nativo de Python.

El método `.tolist()` tiene la función de transformar el resultado obtenido por el método `.columns()` en una lista, y en general, de transformar cualquier serie o DataFrame proveniente de pandas en una lista de Python.

Luego, el método `.join()` tiene la función de concatenar cadenas de caracteres de una lista en Python para convertirlas luego en una sola cadena de caracteres. A la hora de expresar la cadena de caracteres final usa un separador (que para nuestro caso es ", ") y así poder mostrar cada elemento de la lista de manera ordenada.

```
In [ ]: print("Las columnas son: ",',', '.join(df.columns.tolist()),",",sep="")
```

Las columnas son: año, secretaria, nombreestablecimiento, zona, direccion, telefono, nombre_Rector, tipo_Establecimiento, etnias, sector, genero, niveles, jornadas, caracter, especialidad, grados, modelos_Educativos, capacidades_Excepcionales, discapacidades, idiomas, numero_de_Sedes, estado, prestador_de_Servicio, propiedad_Planta_Fisica, resguardo, matricula_Contratada, calendario, internado, estrato_Socio_Economico, correo_Electronico.

En el caso de las filas, para conocer cuántas filas hay se puede usar simplemente el método `.shape` el cual nos entrega las filas y columnas en el siguiente formato: (filas,columnas). Como lo que interesa conocer son

solamente las filas, usando `.shape[0]` nos muestra el primer valor del formato que se presentó antes, es decir, solamente las filas.

```
In [ ]: print("La cantidad de filas es: ",df.shape[0],".",sep="")
```

La cantidad de filas es: 2404.

2. Seleccione los colegios donde la propiedad de la planta física es de una persona natural o de una comunidad religiosa.

Para identificar si la propiedad de la planta física de los colegios es de una persona natural o de una comunidad religiosa se puede usar el método `.query()`, el cual nos ayuda en clasificar aquellas filas las cuales posean ciertas características o cumplan ciertas condiciones. En este caso las condiciones que deben cumplir es que `propiedad_Planta_Fisica == "PERSONA NATURAL"` o que `propiedad_Planta_Fisica == "COMUNIDAD RELIGIOSA"`. El DataFrame final se guarda en el objeto `df_propiedad_Planta_Fisica` y se imprime en pantalla.

```
In [ ]: df_propiedad_Planta_Fisica = df.query('propiedad_Planta_Fisica == "PERSONA NATURAL" | propiedad_Planta_Fisica == "COMUNIDAD RELIGIOSA")
print("Los colegios donde la propiedad de la planta física es de una persona natural o de una comunidad religiosa se pueden ver en la siguiente tabla:")
df_propiedad_Planta_Fisica
```

Los colegios donde la propiedad de la planta física es de una persona natural o de una comunidad religiosa se pueden ver en la siguiente tabla:

Out[]:

	año	secretaria	nombreestablecimiento	zona	direccion	telefono	nombre_Rector	tipo_Establ
0	2016	BOGOTA	INST DE BTO TEC COMERCIAL PITAGORAS ...	URBANA	KR 5 11 67	5665677 / 3429092	YOLANDA ARIAS CRUZ	INS E
1	2016	BOGOTA	GIMNASIO INFANTIL FEDERICO FROEBEL	URBANA	CL 32 SUR 52 A 54	7100349	SANDRA MILENA MENDEZ	CENTRO EI
2	2016	BOGOTA	GIMNASIO INFANTIL LOS NIÑOS DEL MAÑANA	URBANA	CL 134 D 47 27	6010350	CLAUDIA BOCACHICA SAENZ	INS E
4	2016	BOGOTA	COL SAN JUAN VIANETH ...	URBANA	KR 2 C 3 18 SUR	2804738 -- 4714934	GLORIA ESPERANZA CHUQUIN LAISECA	CENTRO EI
5	2016	BOGOTA	COL SAN PATRICIO ...	URBANA	KR 80 137 02	6838640 -- 6815357	PAULINA ENCINALES DE SANJINES	INS E
...	...	...	...	...	...	...	...	
2399	2016	BOGOTA	COLEGIO SAN JOSE DE CALASANZ	URBANA	CL 129 88 B 35	6854492 -- 6854495 -- 6803549	GLADIS CUÉLLAR IBÁÑEZ	INS E
2400	2016	BOGOTA	COL NUEVO REINO DE TURINGIA	URBANA	KR 103 B 152 C 34	6830915	RUTH CECILIA CORREA FAJARDO	CENTRO EI
2401	2016	BOGOTA	LIC INF GOOFY ...	URBANA	CL 65 F SUR 78 C 32	7761739 / 7762538 / 7753157 /8045802	SANDRA PATRICIA SANTAMARIA OVALLE	CENTRO EI
2402	2016	BOGOTA	LIC APRENDO CON MIS AMIGOS ...	URBANA	CR. 94 NO.29-30	2671099	YANNETH CONSUELO BOLIVAR GAITAN	CENTRO EI
2403	2016	BOGOTA	JARD INF EL ROSAL DE MI INFANCIA ...	URBANA	TR. 56A NO.45-18 SUR	7242172	ADALGIZA LADINO	CENTRO EI

1878 rows × 30 columns

¿Hay alguno de estos colegios que sea para capacidades excepcionales?

Para poder responder a la pregunta es pertinente primero realizar un análisis de los datos que se encuentran en la columna "capacidades_Excepcionales" del DataFrame previamente creado `df_propiedad_Planta_Fisica` donde nos podemos dar cuenta que se compone de valores nulos (NaN) para aquellos colegios que no son para capacidades excepcionales y algún otro valor para aquellos que sí;

por lo que se filtra aquellos que tienen un valor diferente a NaN usando el método `.notna()` para luego mostrar los resultados.

Ya que el método `notna()` categoriza específicamente un DataFrame (en nuestro caso `df_capacidades_Excepcionales`) en "True" o "False" dependiendo si no es un NaN (True) o si lo es (False), con ayuda del método `.sum()` el cual toma los valores de True = 1, de False = 0 y los suma todos, dando como resultado cuantos colegios son para capacidades excepcionales.

Finalmente, para una mejor muestra de los datos, se usan unos condicionales para identificar si todos los colegios son para capacidades excepcionales o si algunos los son para que los muestre, en dando caso de que ninguno lo sea, también lo enseña; a su vez, con ayuda de un ternario y el método `.empty` se hace la impresión de la base de datos de aquellos colegios que cumplen la condición inicial, siendo que, si `df_capacidades_Excepcionales` está vacío no imprima nada, y si no lo está, imprima lo que tiene.

```
In [ ]: df_capacidades_Excepcionales = df_propiedad_Planta_Fisica['capacidades_Excepcionales'].notna()

if df_capacidades_Excepcionales.any():
    if df_capacidades_Excepcionales.sum() == df_propiedad_Planta_Fisica.shape[0]:
        print("Sí, ¡todos los colegios son para capacidades excepcionales!")
        df_capacidades_Excepcionales = pd.DataFrame()
    else:
        print("Sí, hay un total de",df_capacidades_Excepcionales.sum(),"colegios para capacidades excepcionales.")
        df_capacidades_Excepcionales = df_propiedad_Planta_Fisica.loc[df_capacidades_Excepcionales]
else:
    print("No hay colegios que sean para capacidades excepcionales.")
    df_capacidades_Excepcionales = pd.DataFrame()

pd.set_option('display.max_columns', None)
df_capacidades_Excepcionales if not df_capacidades_Excepcionales.empty else None
```

Sí, hay un total de 7 colegios para capacidades especiales y estos son:

Out[]:

	año	secretaria	nombreestablecimiento	zona	direccion	telefono	nombre_Rector	tipo_Est:
263	2016	BOGOTA	COL CAMPESTRE MAXIMINO POITIERS ...	URBANA	KR 92 148 51	6835013 / 6841539 ...	KILIAN ALEXANDER ARENAS SANABRIA	
507	2016	BOGOTA	LIC VAL (VIDA-AMOR- LUZ) ...	URBANA	CL 71 C 51 31	2251109 -- 2401331 -- 2502167	LUZ ESPERANZA MORALES SARMIENTO	
1012	2016	BOGOTA	LIC ECOLOGICO DEL NORTE ...	URBANA	CR. 98A NO.156D- 26	6822591 / 6839957 ...	JULIA ALICIA PELAEZ CASTILLO	
1367	2016	BOGOTA	GIMNASIO CAMPESTRE STEPHEN HAWKING	URBANA	KR 68 173 A 50	6718575 -- 3005581260	DORA ASTRID PARDO AVILA	
1964	2016	BOGOTA	COL JONATHAN SWIFT ...	URBANA	KR 90 163 30	6818191	DIANA J. MARQUEZ S.	
2282	2016	BOGOTA	COL CALATRAVA	URBANA	KR 80 156 95	6847860	CONSUELO VALDERRAMA MORALES	
2341	2016	BOGOTA	INST DE PEDAG AUTOACTIVA DE GRUPOS (IPAG) ...	URBANA	KR 8 C 183 32	6711029 -- 6701198	CARMEN TULIA CORDOBA ACOSTA	

¿Cuáles son los colegios que ofrecen todos los niveles educativos y qué idiomas ofrecen?

Para conocer cuáles son los colegios que ofrecen todos los niveles educativos se parte de una suposición la cual es: *"los colegios que, en principio, ofrecen todos los niveles educativos deben ser aquellos que ofrecen más niveles educativos en comparación al resto de la base de datos"*. Esta suposición se hace debido a que fuera de la base de datos, no se tiene ninguna otra información de cuáles son todos los niveles educativos para poder crear un filtro correspondiente.

Haciendo caso a la suposición, para conocer el número máximo de palabras en la columna "niveles" se cambia el tipo de dato de la columna por `str` o "string" (texto) usando el método `.astype()`, luego con ayuda del método `.str.split()` el cual divide el contenido de cada celda de la columna "niveles" en una lista usando como separador identificador "," y con ayuda del método `.apply()` el cual aplica la misma función a cada elemento de una serie, que para este caso la función sería `len` la cual nos daría la longitud de cada lista generada por `.str.split()` para cada fila, se obtiene una nueva columna llamada "max_palabras" en donde se guarda la cantidad de palabras de cada fila para la columna "niveles".

Posteriormente, reescribiendo en un nuevo DataFrame llamado `df_niveles` aquellas filas que cumplen la condición `df['max_palabras'] == df['max_palabras'].max()` (el comando `.max()` sirve para conocer el valor más grande en una serie), se imprime dicho DataFrame eliminando la nueva columna creada usando el comando `.drop()`.

Ya que se muestran todas las columnas del DataFrame original solo que filtrado para aquellos colegios que ofrecen todos los niveles educativos, **ahí mismo se puede consultar cuáles son los idiomas que ofrecen.**

```
In [ ]: df['niveles'] = df['niveles'].astype(str)

df['max_palabras'] = df['niveles'].str.split(',').apply(len)

df_niveles = df[df['max_palabras'] == df['max_palabras'].max()]

print("Los colegios que ofrecen todos los niveles educativos son:\n")

df_niveles.drop('max_palabras', axis=1)
```

Los colegios que ofrecen todos los niveles educativos son:

```
Out[ ]:
```

	año	secretaria	nombreestablecimiento	zona	direccion	telefono	nombre_Rector	ti
5	2016	BOGOTA	COL SAN PATRICIO ...	URBANA	KR 80 137 02	6838640 -- 6815357	PAULINA ENCINALES DE SANJINES	
1031	2016	BOGOTA	INSTITUTO JUAN PABLO II	URBANA	DG 71 C BIS SUR 77 H 17	7757668	AMPARO BENAVIDES ALBARRACIN	
1065	2016	BOGOTA	COLEGIO TOMAS CARRASQUILLA (IED)	URBANA	CL 74 A 63 04	6603161 -- 2407959 -- 2400101	ROSA ADELINA RODRIGUEZ RODRIGUEZ	
1450	2016	BOGOTA	COLOMBO BRITANICO - BILINGÜE INTERNACIONAL	URBANA	IND AEROPUERTO GUAYMARAL COSTADO SUR OCCIDENTAL	6683999	WALTER ABONDANO MIKAN	
2236	2016	BOGOTA	COLEGIO NUEVO HORIZONTE (IED)	RURAL,URBANA	KR 5 B 189 B 30	6793861 - 6796685	HAROLD RODRIGO MURILLO TOVAR	

Si bien es cierto otra solución podría darse usando el comando `.unique()` e identificando cual es aquel valor con el mayor número de palabras para luego colocarlo como filtro, esta opción obliga realizar un doble proceso manual para cada base de datos que se quiera usar, y pues, en principio el código presentado ofrecería una identificación automática para cada base de datos donde simplemente hay que variar la base de datos y la columna por filtrar.

Ejercicio 2.

En este ejercicio se usará la librería `pyeph`, la cual es una librería personalizada de Python creada por el Instituto Nacional de Estadísticas y Censos de la república de Argentina cuyo fin es el procesamiento en Python de las Encuestas Permanentes de Hogares (eph) publicadas de forma periódica. Es un espacio donde

se centralizan los cálculos vinculados a las mismas para posteriormente ser utilizadas en investigaciones, artículos, publicaciones, entre otros.

Primero es pertinente importar la librería pero antes hay que instalarla. Usando el comando `$ pip install pyeph` desde la terminal de la máquina que ya tenga previamente instalado Python, se instala la librería para que posteriormente usando `import pyeph`, quede importada dentro de nuestro código.

```
In [ ]: import pyeph
```

Tratamiento de las bases de datos

Primero hay que importar la base de datos correspondiente al segundo trimestre del 2021, eso se hace mediante el comando `pyeph.obtener(data="eph", year=2021, period=2, base_type='individual')`, esta quedará guardada en el objeto `eph`.

La forma en que la librería importa la base de datos lo hace por medio de un archivo `.zip` que descarga y coloca en una carpeta llamada "pyeph" la cual estará ubicada en la misma ruta del archivo `.ipynb` principal.

```
In [ ]: eph = pyeph.obtener(data="eph", year=2021, period=2, base_type='individual')
```

Obtenido con éxito: base_individual_2021T2.zip

Luego hay que identificar las columnas, para ello se usará exactamente el mismo comando usado en el *Ejercicio 1* solo que cambiando la base de datos.

```
In [ ]: print("Las columnas son: ",', '.join(eph.columns.tolist()),",",sep="")
```

Las columnas son: CODUSU, ANO4, TRIMESTRE, NRO_HOGAR, COMPONENTE, H15, REGION, MAS_500, AGLOMERAD O, PONDERA, CH03, CH04, CH05, CH06, CH07, CH08, CH09, CH10, CH11, CH12, CH13, CH14, CH15, CH15_CO D, CH16, CH16_COD, NIVEL_ED, ESTADO, CAT_OCUP, CAT_INAC, IMPUTA, PP02C1, PP02C2, PP02C3, PP02C4, PP02C5, PP02C6, PP02C7, PP02C8, PP02E, PP02H, PP02I, PP03C, PP03D, PP3E_TOT, PP3F_TOT, PP03G, PP0 3H, PP03I, PP03J, INTENSI, PP04A, PP04B_COD, PP04B1, PP04B2, PP04B3_MES, PP04B3_ANO, PP04B3_DIA, PP04C, PP04C99, PP04D_COD, PP04G, PP05B2_MES, PP05B2_ANO, PP05B2_DIA, PP05C_1, PP05C_2, PP05C_3, PP05E, PP05F, PP05H, PP06A, PP06C, PP06D, PP06E, PP06H, PP07A, PP07C, PP07D, PP07E, PP07F1, PP07F 2, PP07F3, PP07F4, PP07F5, PP07G1, PP07G2, PP07G3, PP07G4, PP07G_59, PP07H, PP07I, PP07J, PP07K, PP08D1, PP08D4, PP08F1, PP08F2, PP08J1, PP08J2, PP08J3, PP09A, PP09A_ESP, PP09B, PP09C, PP09C_ES P, PP10A, PP10C, PP10D, PP10E, PP11A, PP11B_COD, PP11B1, PP11B2_MES, PP11B2_ANO, PP11B2_DIA, PP11 C, PP11C99, PP11D_COD, PP11G_ANO, PP11G_MES, PP11G_DIA, PP11L, PP11L1, PP11M, PP11N, PP11O, PP11 P, PP11Q, PP11R, PP11S, PP11T, P21, DECOCUR, IDECOCUR, RDECOCUR, GDECOCUR, PDECOCUR, ADECOCUR, PO NDIIO, TOT_P12, P47T, DECINDR, IDECINDR, RDECINDR, GDECINDR, PDECINDR, ADECINDR, PONDII, V2_M, V3 _M, V4_M, V5_M, V8_M, V9_M, V10_M, V11_M, V12_M, V18_M, V19_AM, V21_M, T_VI, ITF, DECIFR, IDECIF R, RDECIFR, GDECIFR, PDECIFR, ADECIFR, IPCF, DECCFR, IDECCFR, RDECCFR, GDECCFR, PDECCFR, ADECCFR, PONDIIH.

Finalmente, conociendo cuáles son las columnas de la base de datos, se crea un filtro en dónde se almacenan solamente las columnas "CH06,CH04,P21,CAT_OCUP,ESTADO,PONDERA" para luego trabajar con ellos. Como un añadido se les cambió su nombre luego de hacer la copia para una mejor interpretación de que es lo que representa cada una, para eso se usó el método `.columns`.

```
In [ ]: eph_2 = eph[['CH06','CH04','P21','CAT_OCUP','ESTADO','PONDERA']]
eph_2.columns = ['EDAD','SEXO','INGRESOS','OCUPACIÓN','ESTADO','PONDERACIÓN']
```

Traiga un DataFrame que filtre solo a las personas que tengan 21 años o menos

Para ello, se usa el método `.query()` donde el filtro será `EDAD <= 21` . Luego se imprime el DataFrame resultante.

```
In [ ]: eph_2_21anos = eph_2.query('EDAD <= 21')
eph_2_21anos
```

```
Out[ ]:
```

	EDAD	SEXO	INGRESOS	OCUPACIÓN	ESTADO	PONDERACIÓN
4	18	1	0	0	3	192
8	5	1	0	0	4	250
11	19	2	0	0	3	250
12	17	1	0	0	3	250
13	14	1	0	0	3	250
...	...	...	...	...	...	...
47070	15	2	0	0	3	46
47071	13	1	0	0	3	46
47074	8	1	0	0	4	49
47077	10	1	0	0	3	111
47084	20	1	4000	2	1	230

15192 rows × 6 columns

Traiga un DataFrame que filtre solo a las personas "desocupadas" que tengan más de 40 años

Para ello, se usa el método `.query()` donde el filtro será `EDAD > 40 and OCUPACIÓN == 2` . Luego se imprime el DataFrame resultante.

```
In [ ]: eph_2_40anos = eph_2.query('EDAD > 40 and OCUPACIÓN == 2')
eph_2_40anos
```


Out[]:

	EDAD	SEXO	INGRESOS	OCUPACIÓN	ESTADO	PONDERACIÓN
0	58	1	50000	2	1	214
36	60	1	70000	2	1	1445
37	61	2	70000	2	1	1445
48	49	1	32000	2	1	8647
60	41	1	-9	2	1	105
...	...	...	...	...	...	...
46998	41	1	60000	2	1	228
47039	59	1	-9	2	1	541
47046	54	1	-9	2	1	504
47051	43	2	-9	2	1	225
47081	57	1	-9	2	1	86

2679 rows × 6 columns

¿Cuántas mujeres hay en cada base de datos y cuántos hombres?

Para responder a esta pregunta se usa el método `.query()` donde los filtros serán dados por la columna "SEXO" siendo que si es igual a 1 es hombre y si es igual a 2 es mujer. Luego, con el método `.SEXO.count()` cuenta cuantas personas cumplieron el filtro y las muestra en pantalla, así para DataFrame hecho previamente.

Como un añadido, muestra también el total de personas que hay en el primer y segundo DataFrame para realizar una comparativa de los resultados.

```
In [ ]: print("Hay un total de ", eph_2_21anos.query('SEXO == 2').SEXO.count(), " mujeres y un total de ",  
print()  
print("Hay un total de ", eph_2_40anos.query('SEXO == 2').SEXO.count(), " mujeres y un total de ",
```

Hay un total de 7438 mujeres y un total de 7754 hombres en el primer DataFrame. El total de personas en el primer DataFrame es: 15192.

Hay un total de 983 mujeres y un total de 1696 hombres en el segundo DataFrame. El total de personas en el segundo DataFrame es: 2679.

¿Cuál es la tasa de desocupados en el periodo seleccionado para las mujeres?, ¿es más alta o más baja que la de los hombres?

Para calcular la tasa de "desocupados" en general, se crea una función llamada `tasa_desocupados` la cual pide los valores de dos variables, `desocupados` y `ocupados`, para luego operarlas y obtener la tasa de "desocupados".

```
In [ ]: def tasa_desocupados(desocupados, ocupados):  
return desocupados / (ocupados + desocupados)
```

Así, guardando la cantidad de "desocupados" y "ocupados" para hombres y mujeres de cada base de datos en una variable independiente para cada uno (todo esto con ayuda de los métodos ya vistos `.query()` y

.ESTADO.count()) se calcula la tasa para hombres y mujeres de cada base de datos usando la función `tasa_desocupados` ya creada y se compara con un sencillo condicional. Finalmente se imprime en pantalla los resultados obtenidos.

```
In [ ]: m21_desocupado = eph_2_21anos.query('SEXO and ESTADO == 2').ESTADO.count()
m21_ocupado = eph_2_21anos.query('SEXO == 2 and ESTADO == 1').ESTADO.count()
h21_desocupado = eph_2_21anos.query('SEXO == 1 and ESTADO == 2').ESTADO.count()
h21_ocupado = eph_2_21anos.query('SEXO and ESTADO == 1').ESTADO.count()

m40_desocupado = eph_2_40anos.query('SEXO and ESTADO == 2').ESTADO.count()
m40_ocupado = eph_2_40anos.query('SEXO == 2 and ESTADO == 1').ESTADO.count()
h40_desocupado = eph_2_40anos.query('SEXO == 1 and ESTADO == 2').ESTADO.count()
h40_ocupado = eph_2_40anos.query('SEXO and ESTADO == 1').ESTADO.count()

if tasa_desocupados(m21_desocupado,m21_ocupado) > tasa_desocupados(h21_desocupado,h21_ocupado):
    print("La tasa de desocupados de las mujeres, la cual es",round(tasa_desocupados(m21_desocupado,m21_ocupado),4))
else:
    print("La tasa de desocupados de las mujeres, la cual es",round(tasa_desocupados(h21_desocupado,h21_ocupado),4))
print()
if tasa_desocupados(m40_desocupado,m40_ocupado) > tasa_desocupados(h40_desocupado,h40_ocupado):
    print("La tasa de desocupados de las mujeres, la cual es",round(tasa_desocupados(m40_desocupado,m40_ocupado),4))
else:
    print("La tasa de desocupados de las mujeres, la cual es",round(tasa_desocupados(h40_desocupado,h40_ocupado),4))
```

La tasa de desocupados de las mujeres, la cual es 0.35628 aproximadamente, es más alta que la de los hombres para la primera base de datos.

La tasa de desocupados de las mujeres, la cual es 0.12038 aproximadamente, es más alta que la de los hombres para la segunda base de datos.